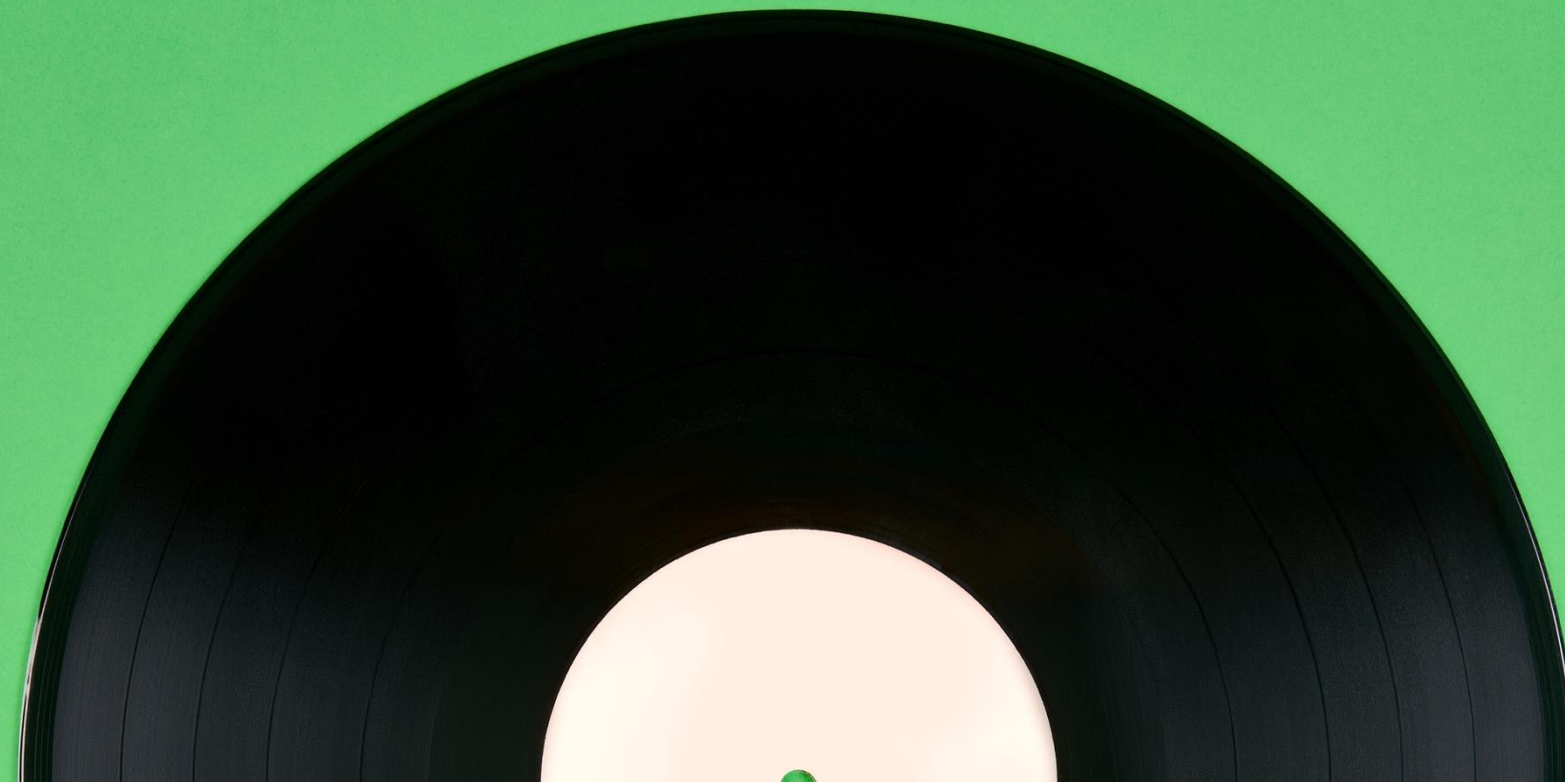
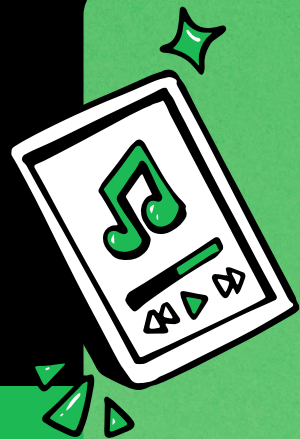




Spotify Song Recommender System using Machine Learning





Areas covered:

Dataset Selected

Exploratory Data Analysis

Data Preprocessing

Feature Engineering

Model Design

Model Evaluation

Model Explainability

Bias Analysis

Ethics

Deployment Considerations

Model Demonstration



Background of Spotify

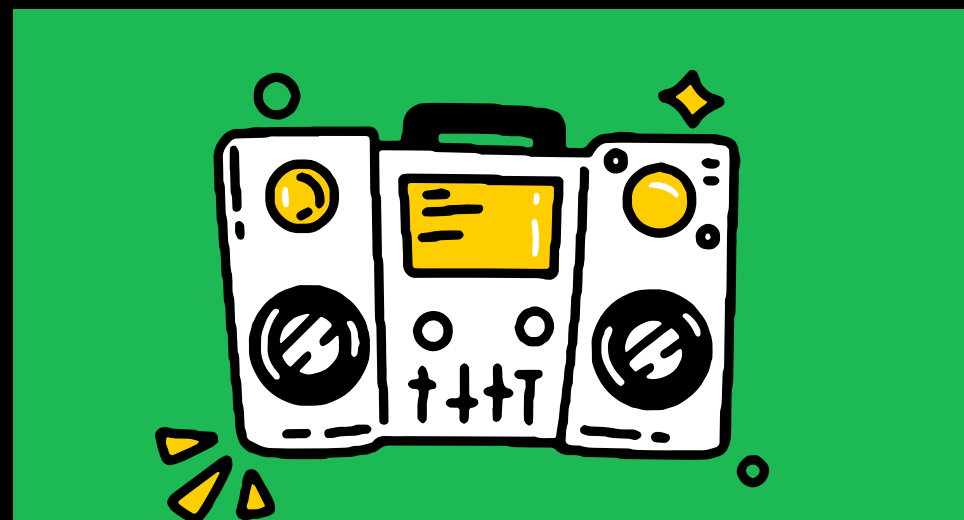


- Spotify hosts over 100m songs, 7m podcast titles, and 700k audiobooks (Spotify, no date).
- Machine Learning (ML) is used to issue song recommendation based on listening data.



Dataset Selection

- 'Dataset selected: 'Spotify Tracks' on Kaggle.
- Metadata of 114k songs across 125 genres, possessing different audio features.
- No user interaction data, so a traditional recommender system is not suitable.
- Content-based recommender system can be developed from it.



Exploratory Data Analysis (EDA)

- 🎧 EDA is crucial – misunderstanding data can result in an inaccurate model that produces misleading insights (Koukaras and Tjortjis, 2025).

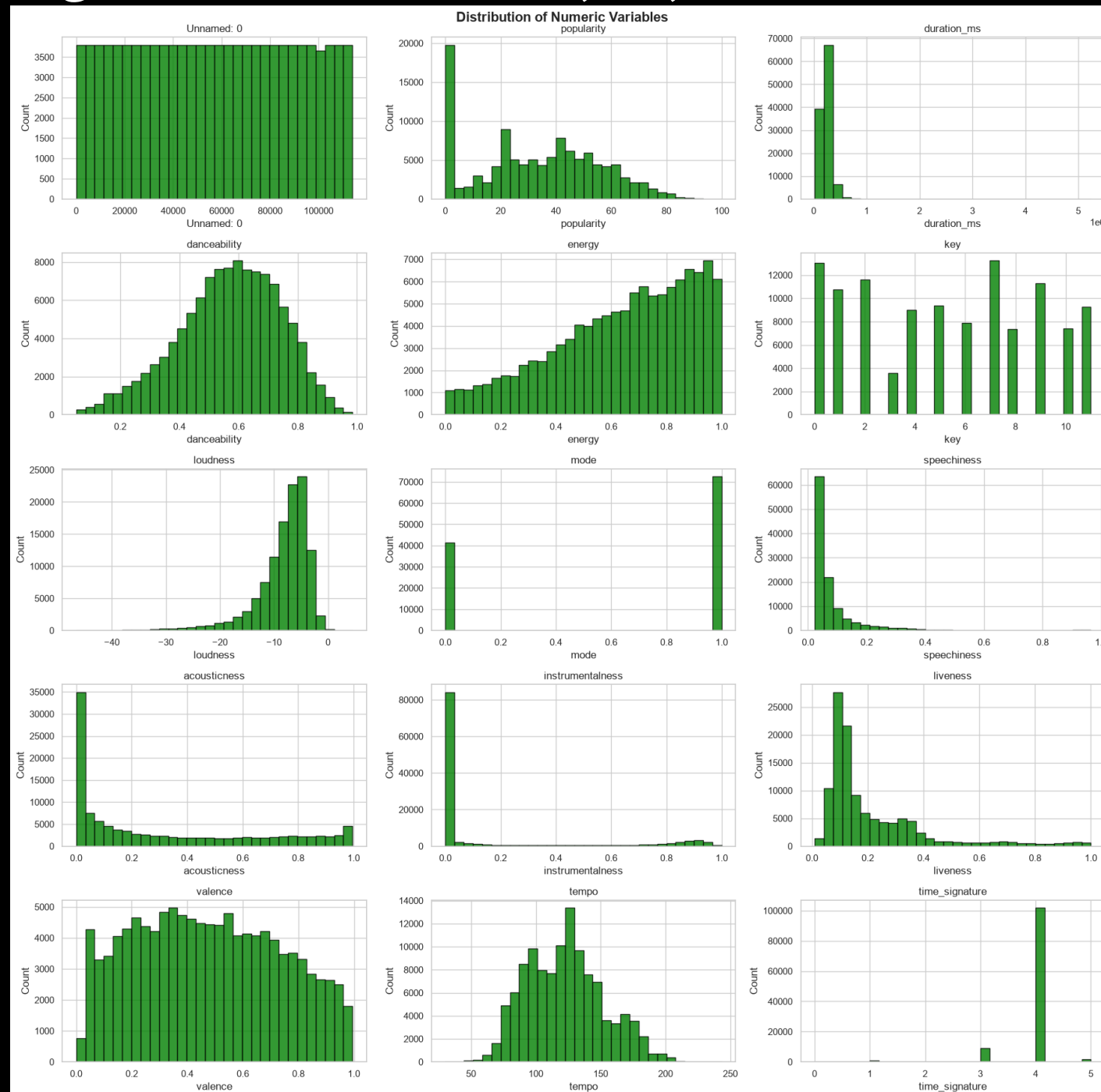


Figure 1: Distributions of Numeric Variables

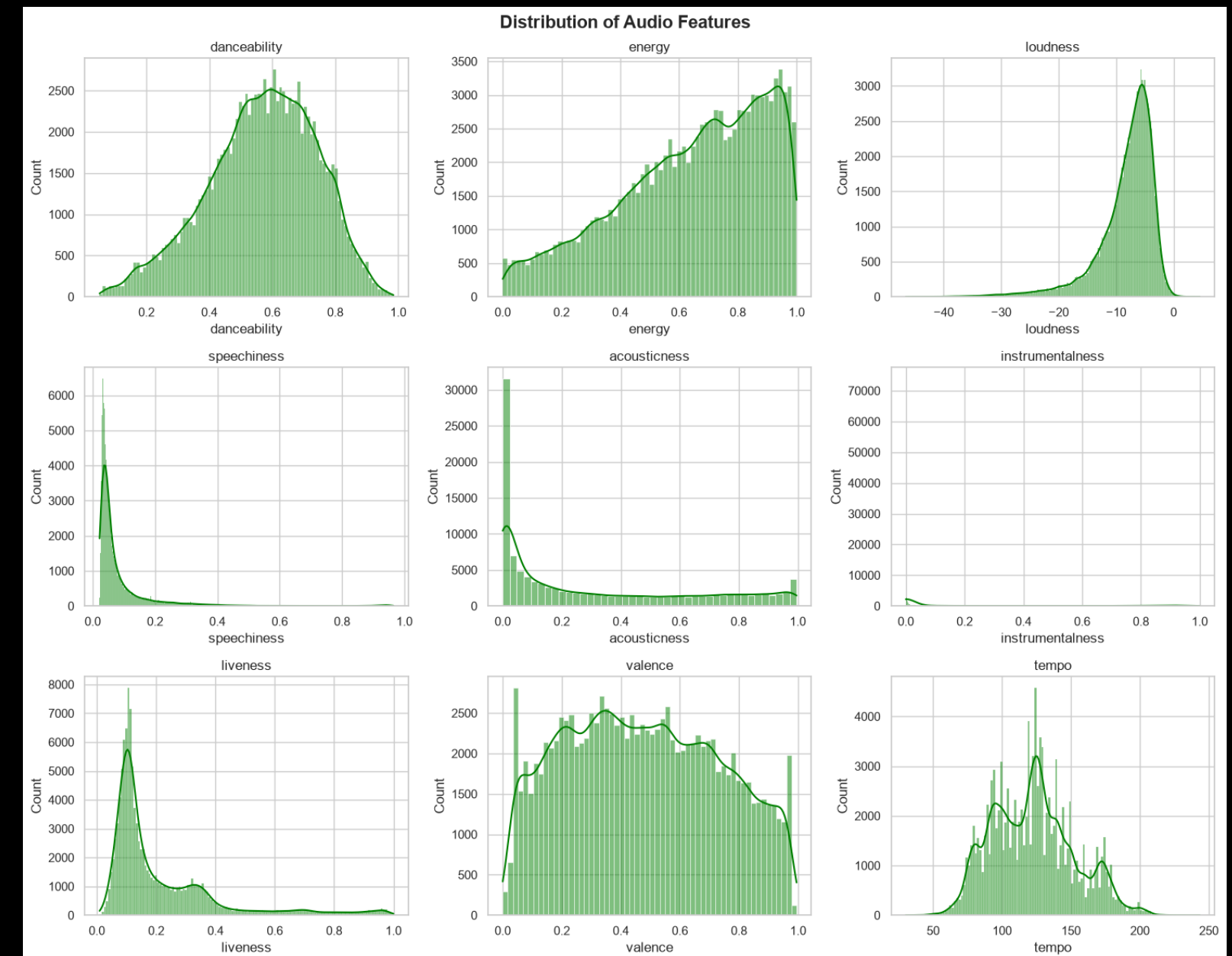


Figure 2: Distributions of Audio Features



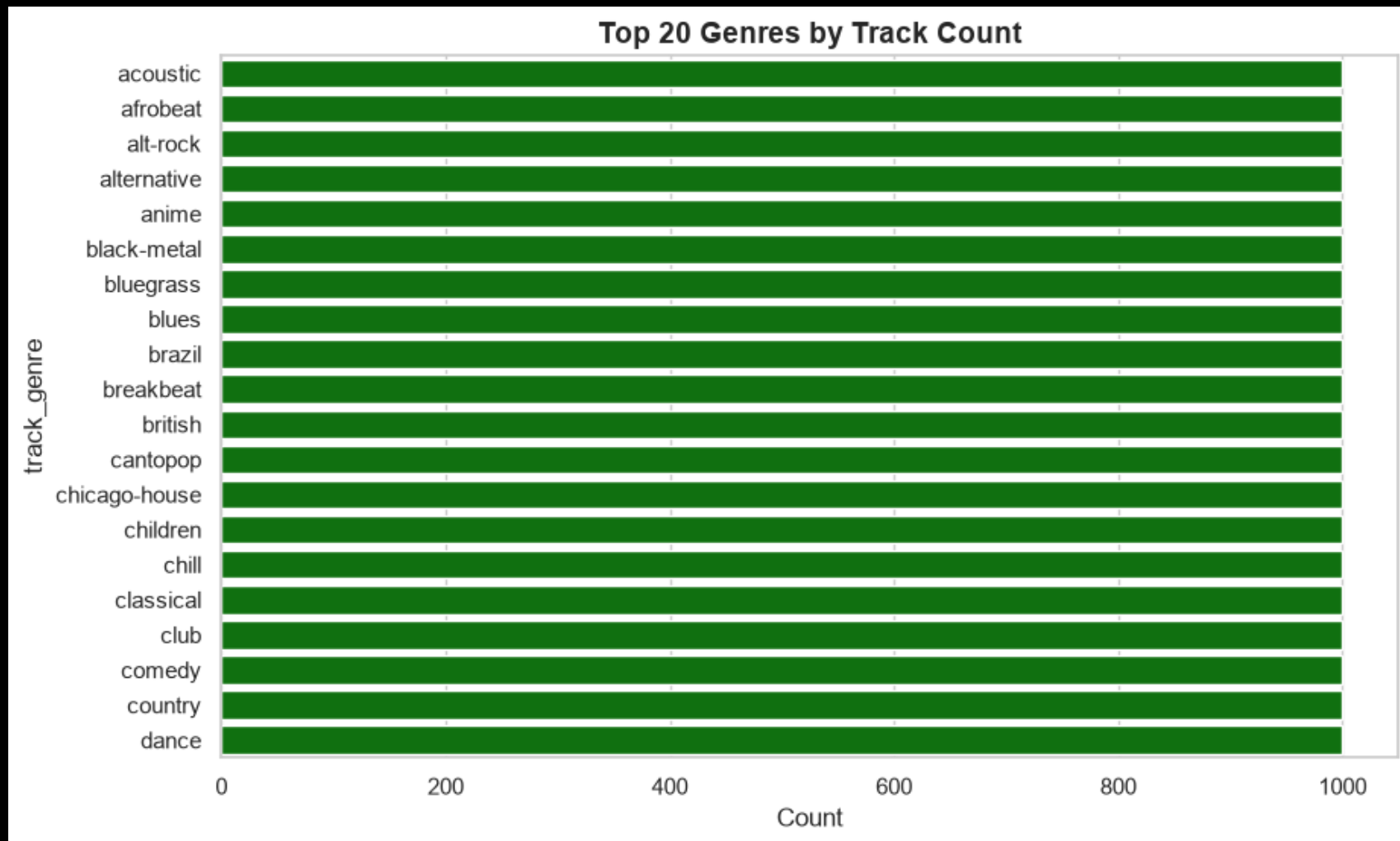


Figure 3: Bar Plot of Top 20 Genres by Track Count

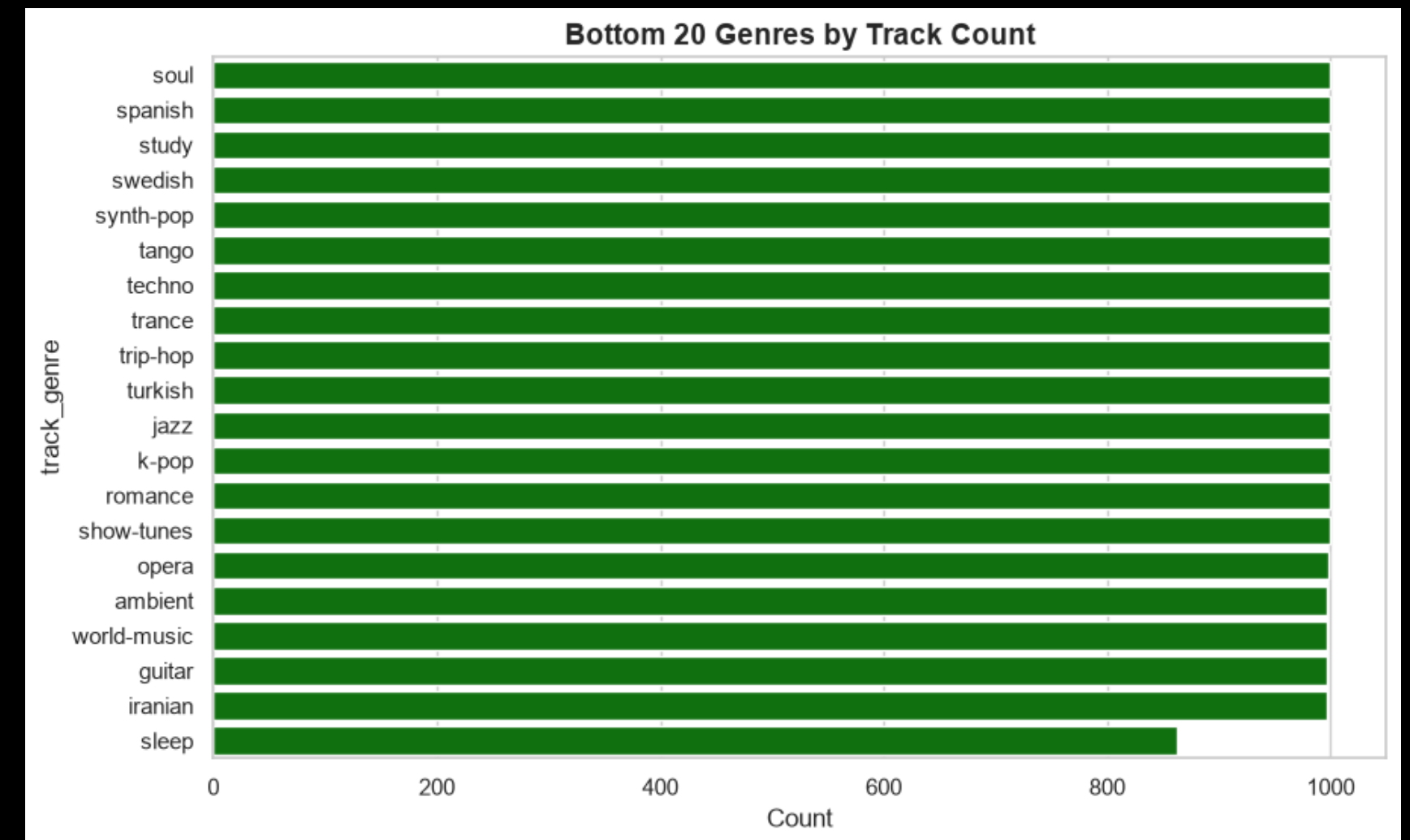


Figure 4: Bar Plot of Bottom 20 Genres by Track Count



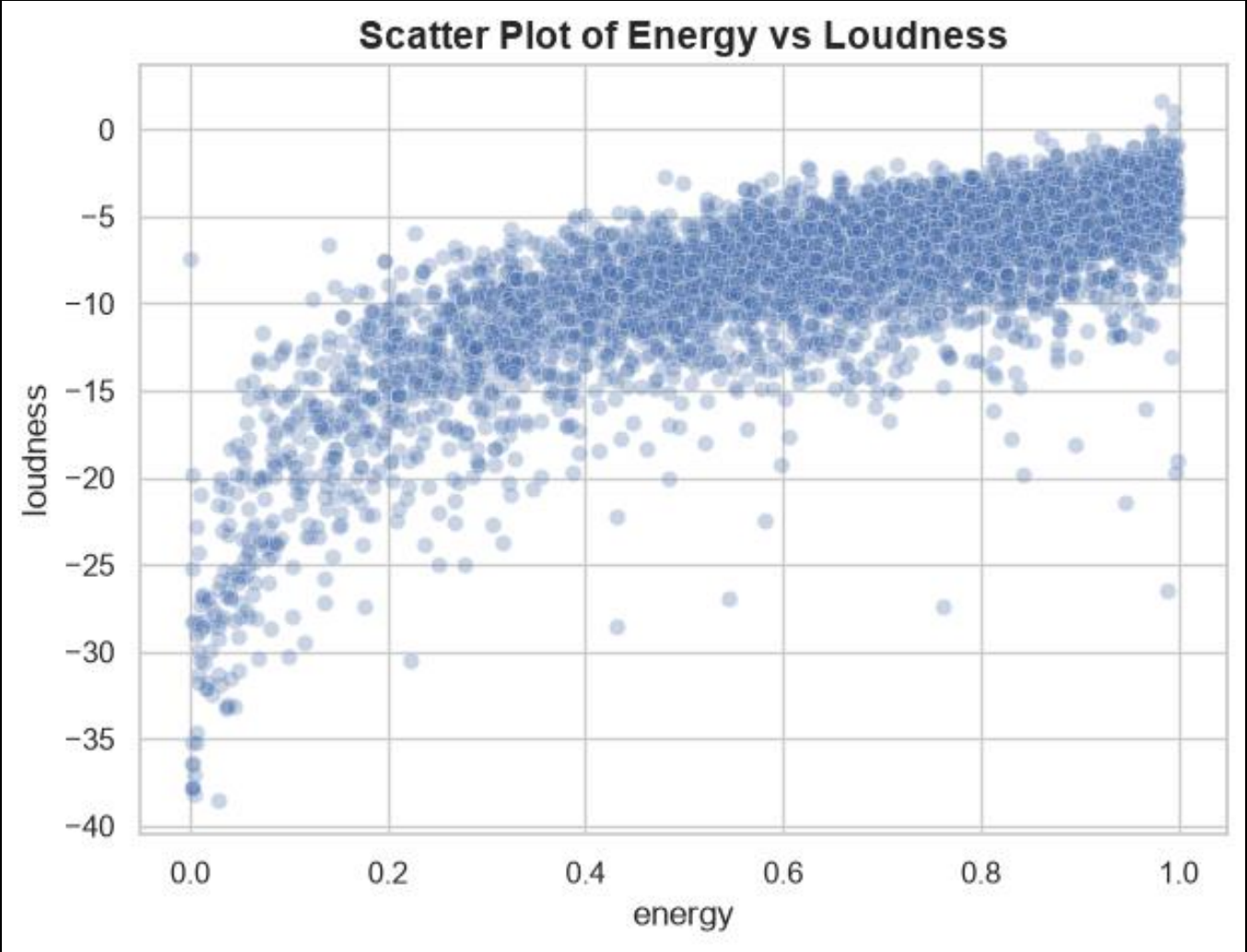
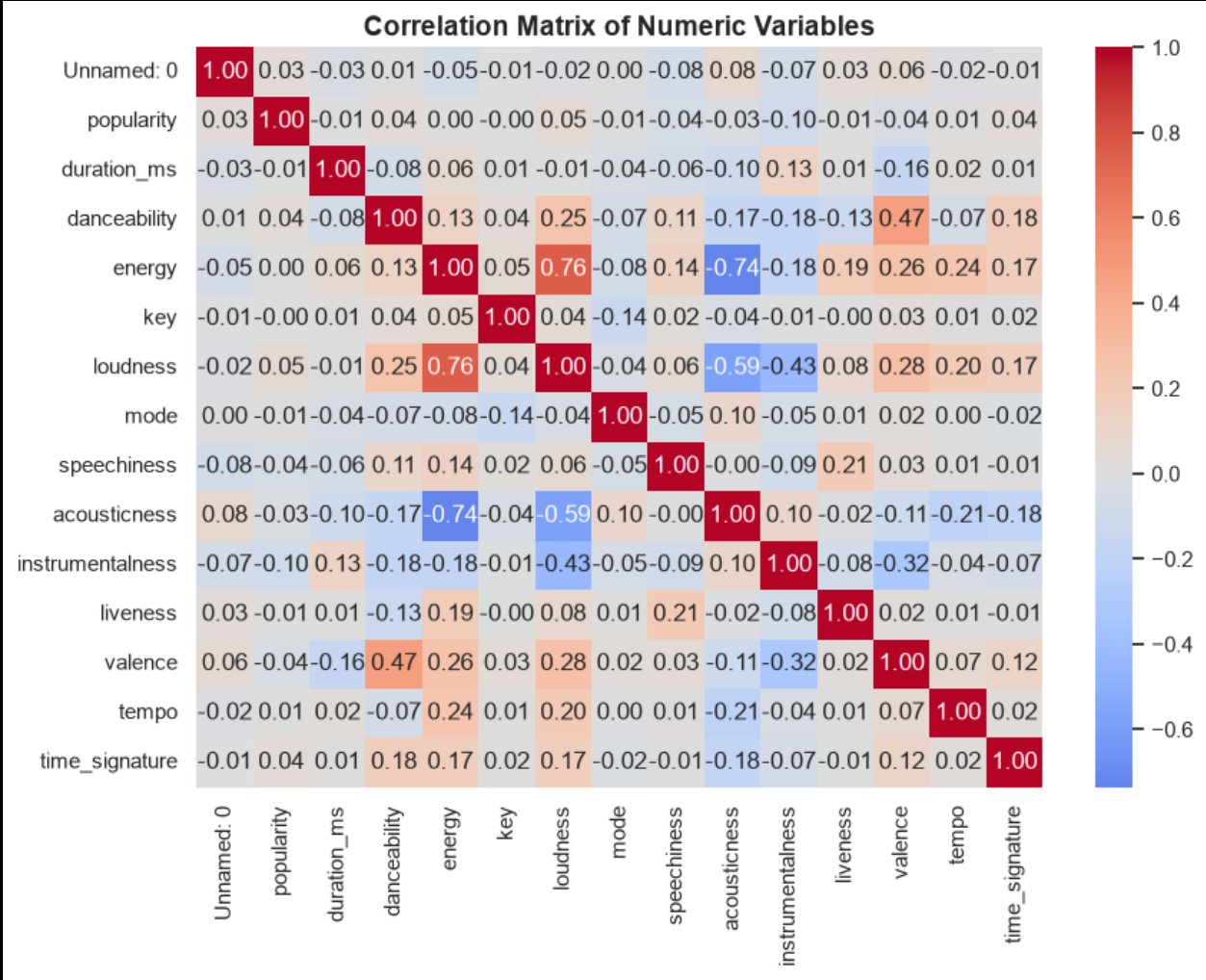


Figure 5: Correlation Matrix of Numeric Variables

Figure 6: Scatter Plot of 'energy' vs 'loudness'

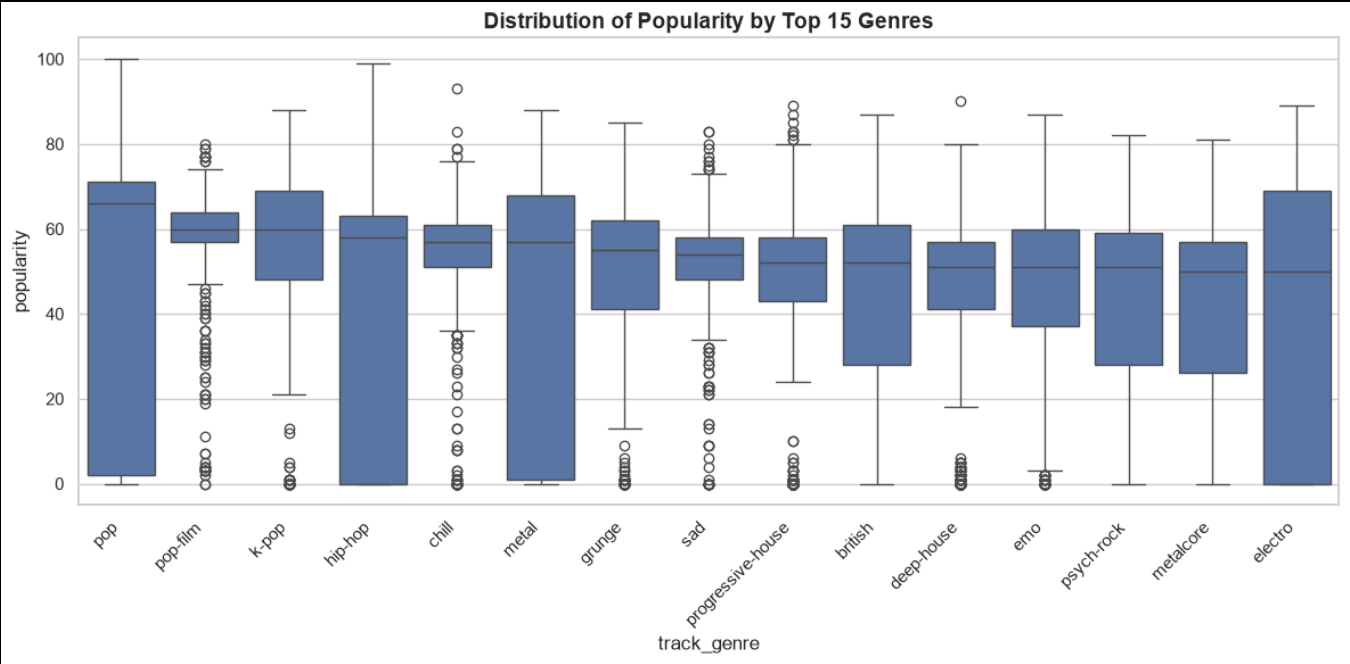


Figure 7: Box Plot Showing Distribution of Popularity by Top 15 Genres



Data Preprocessing



- Duplicated tracks were removed.
- 3 rows with missing fields were dropped.
- Skewed features transformed using Yeo-Johnson to prevent warped learning in neural networks (Japkowicz and Stephen, 2002).

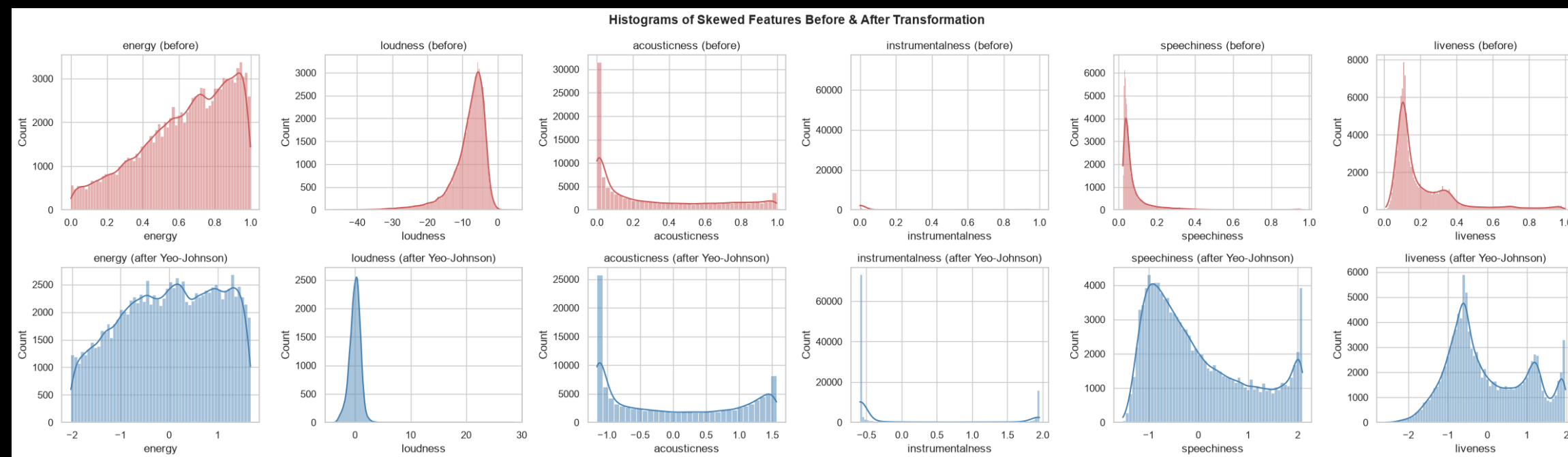


Figure 8: Histograms Displaying Distribution of Skewed Features Before and After Transformation



Feature Engineering



- Genre-frequency feature to separate how 'mainstream' a song is from its popularity.
- 'energy' x 'loudness' and 'danceability' x 'valence' interactions.
- 'genre_match' and 'mood_match' interactions.
- Mood quadrant based on music psychology.



Figure 9: Mood Quadrant Based on 'energy' and 'valence'



Classical ML Model Development



- Chosen model: Random Forest (RF). Allows for non-linear interactions and adapts to high dimensional datasets (Borup et al, 2023).
- Multiple trees were built on engineered features based on specified hyperparameters with their predictions averaged.
- Hyperparameters:
 - Number of trees = 200
 - Max depth = none
 - Minimum samples per leaf = 1

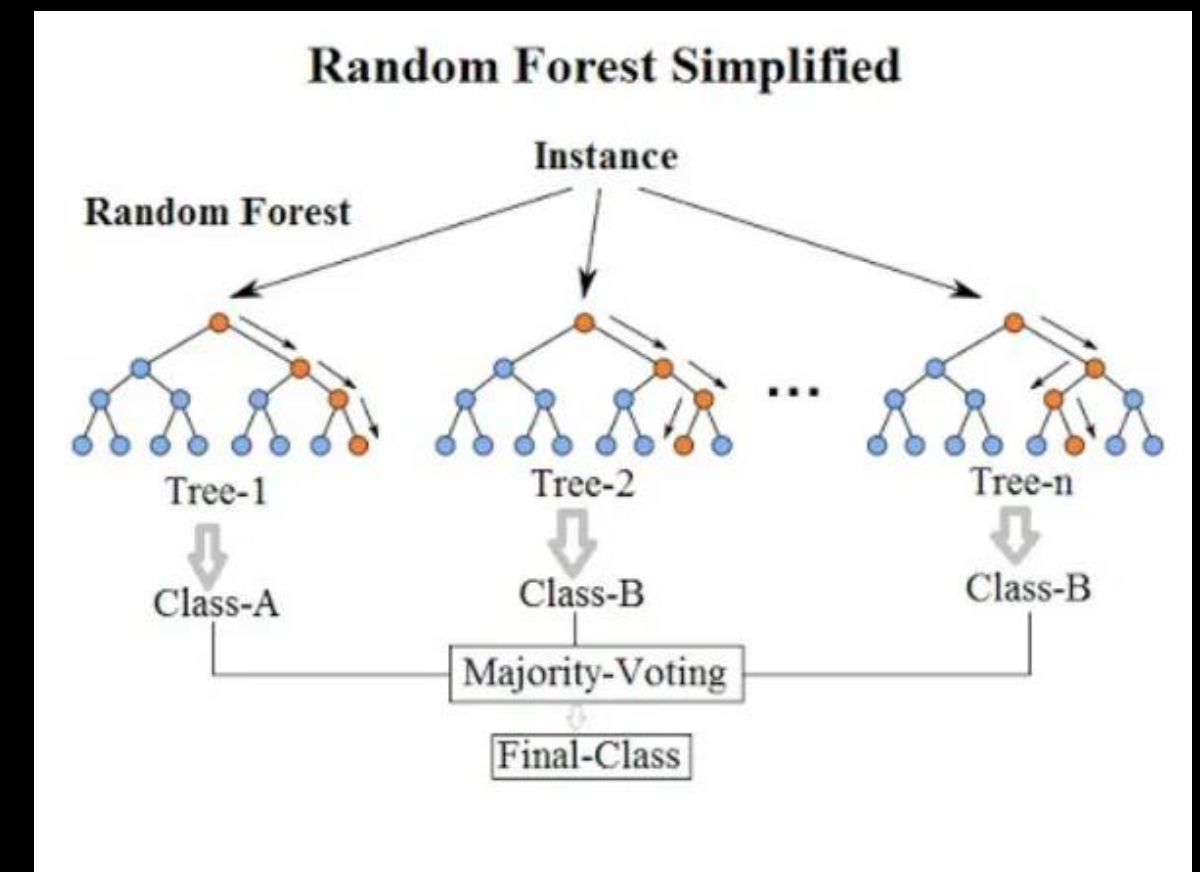


Figure 10: Diagram of Random Forest (Koehrsen, 2020)



Deep Learning Model Development

- Chosen model: Multilayer Perceptron (MLP), an ANN where the data flows in one direction (Sazli, 2006).



- Structure

- Input = audio features + user preferences
- Hidden layers = network learns combinations and interactions
- Output = relevance score (0 – 1)

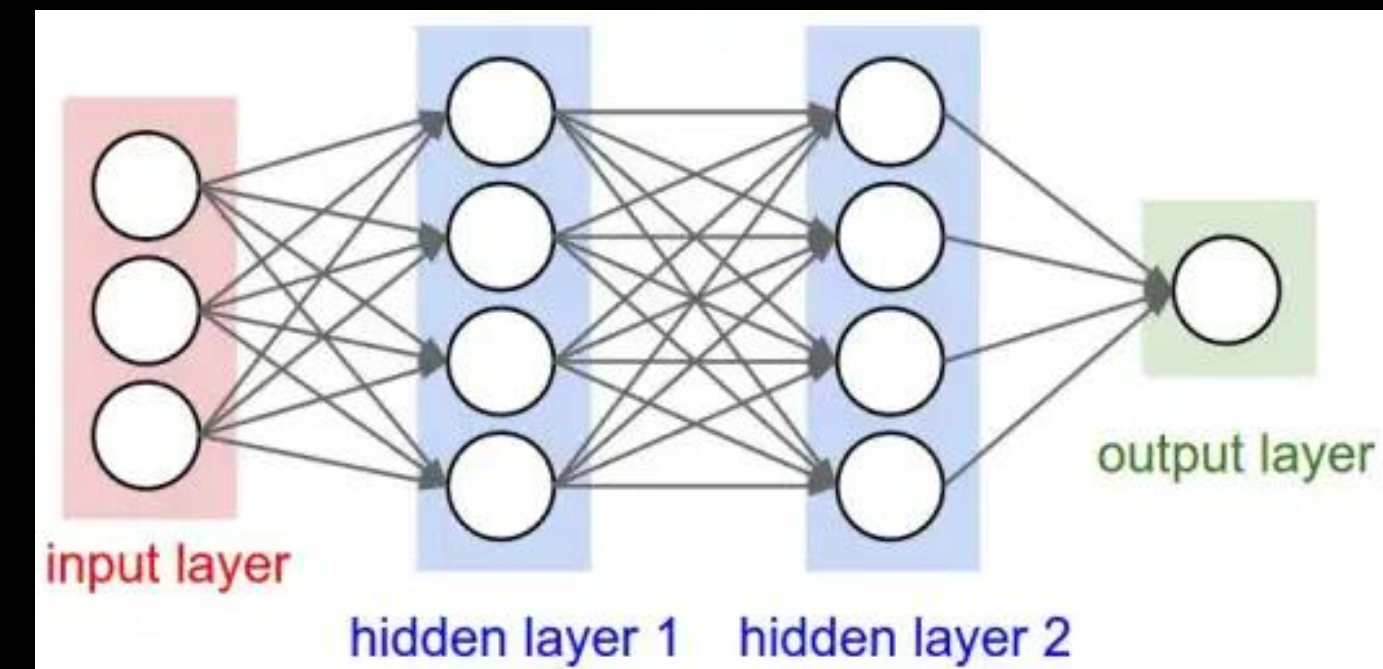


Figure 11: Diagram of a Multilayer Perceptron (Rathi, 2018)

- Hyperparameters:

- 64, 32 units for hidden layers
- Batch normalization and 20% dropout to prevent overfitting
- Adam optimizer used 0.001 learning rate
- 8 epochs



Validation Strategy



- Robust validation required to ascertain the likelihood of an outcome due to chance (Gorriz et al, 2024).
- Model validation using five-fold GroupKFold cross-validation (Kohavi, 1995).
- User profiles grouped with 80/20 train-test split.
- Prevents data leakage and an inflated performance.



Evaluation

🎧 Root Mean Squared Error (RMSE) – measures difference between predicted and true relevance scores (Hodson, 2022).



🎧 Normalised Discounted Cumulative Gain (NDCG) – measures ranking performance (Kshirsagar et al, 2025).

Model	RMSE – mean, std	NDCG – mean, std
RF	0.031, 0.001	0.969, 0.002
MLP	0.034, 0.001	0.969, 0.003

Table 1: Evaluation Metric Scores of Both Models



Explainability with SHAP



- Shapely Additive Explanations (SHAP): an established explainability technique (Lundberg and Lee, 2017).

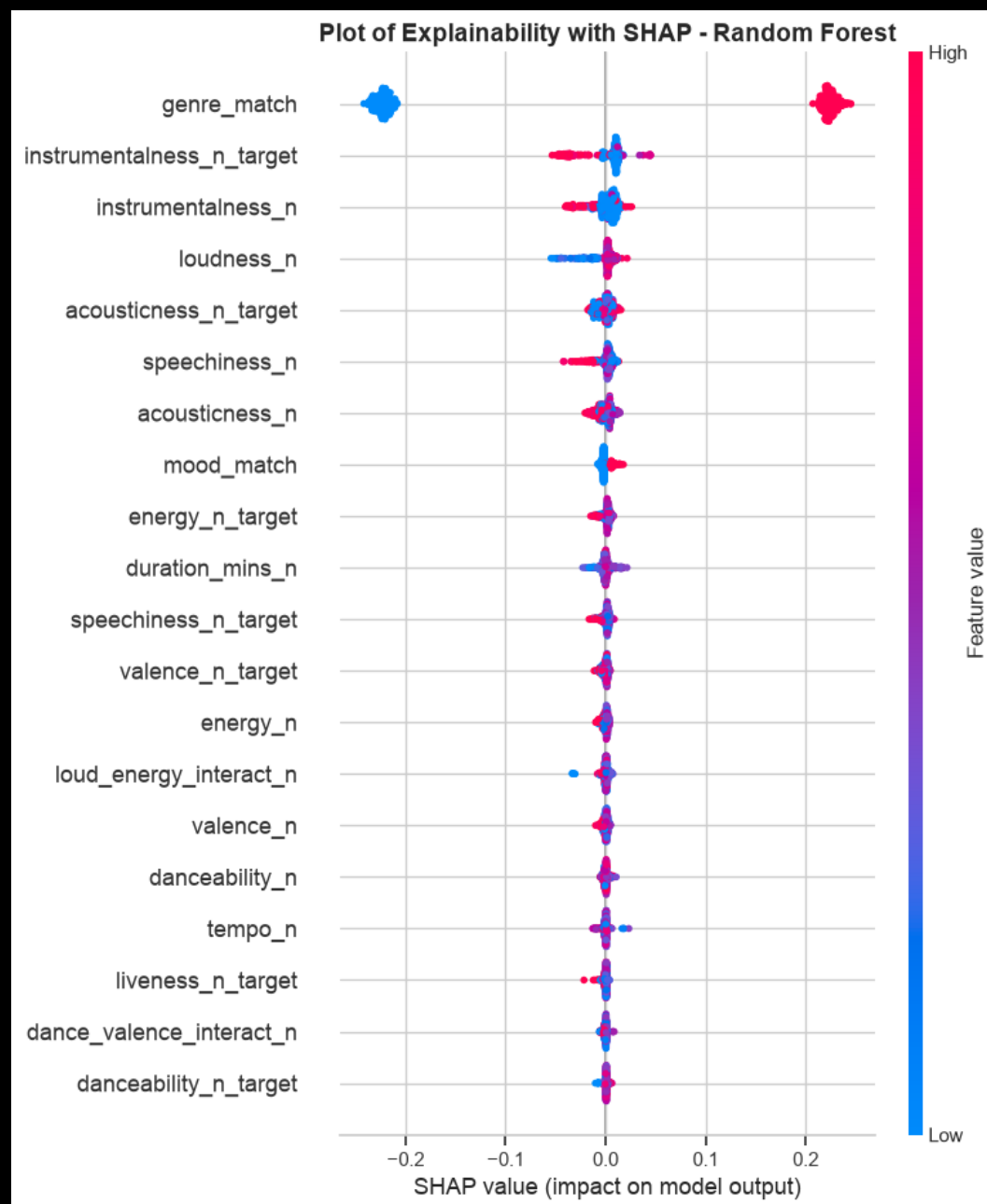


Figure 12: SHAP Summary Plot for RF

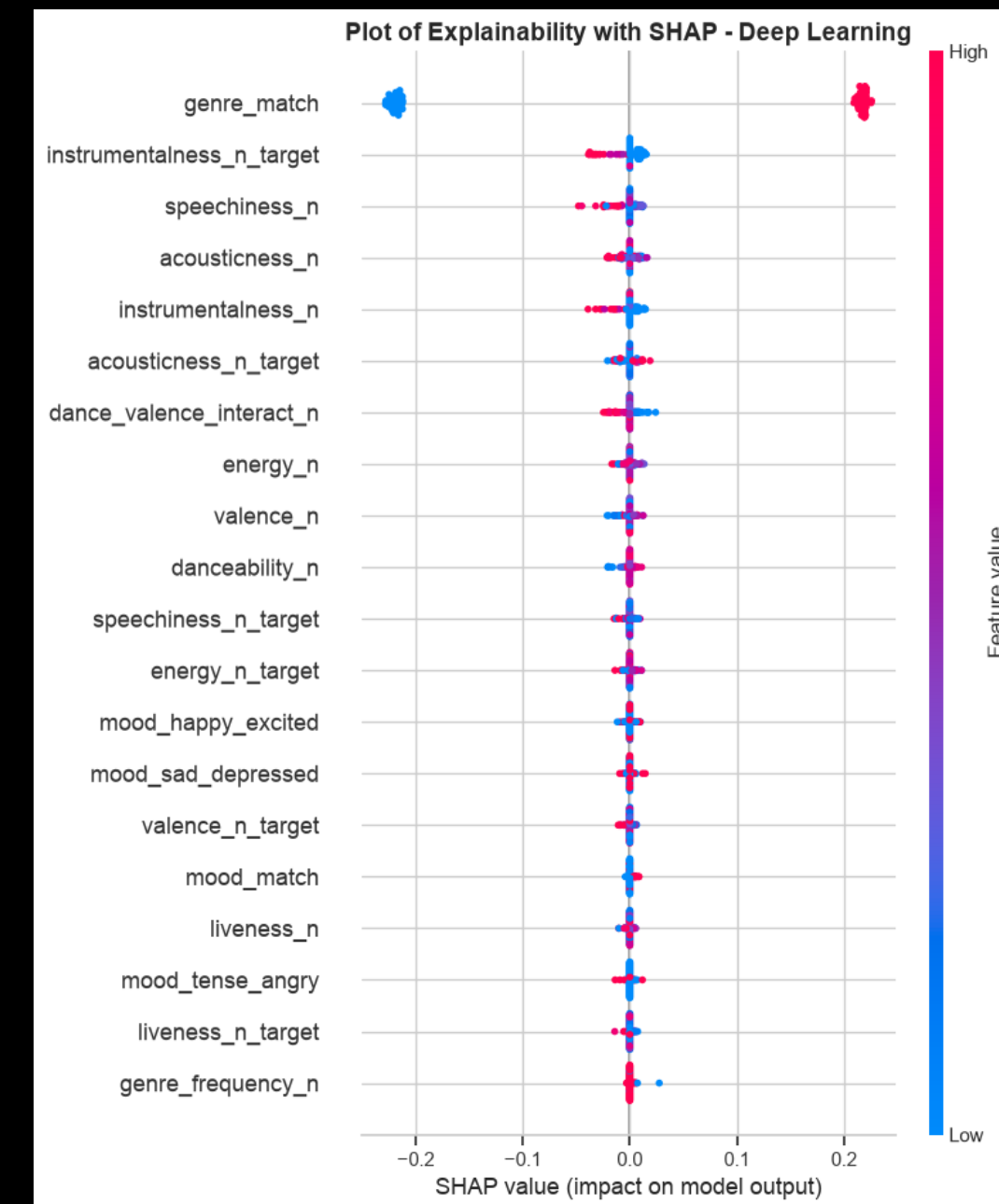


Figure 13: SHAP Summary Plot for MLP



Bias Analysis



🎧 Performance disparities across subgroups were tested to check if low-activity users are underserved.

Segment	RF RMSE	DL RMSE
High Popularity	0.031	0.037
Low Popularity	0.039	0.032
Common Genre	0.035	0.035
Rare Genre	0.035	0.024

Table 2: RMSE Across Popularity and Genre Segments



Ethics



- Data Protection:
 - Dataset features publicly available metadata.
 - Real system requires strict adherence to GDPR policies regarding personal data retention to avoid penalties (Art. 83 GDPR – General conditions for imposing administrative fines, 2018).
- Fairness
 - False negative result = niche track would not be recommended, leading to lost revenue.
 - MLP model would be favoured to mitigate this.



Deployment Considerations



- Trained model placed behind a REST API with a server like Google Vertex AI, ensuring seamless integration with Google Cloud Platform (Spotify Cloud Infrastructure Powers Global Music Streaming, no date).
- Consider using an approximate nearest-neighbour index like Facebook AI Similarity Search (FAISS) due to the volume of songs hosted (Gollapudi et al, 2023).
- Close monitoring of the model's RMSE and NDCG scores plus fairness metrics and rolling retraining.





System demonstration

```
Unit 11

Spotify Recommender.ipynb •
Spotify Assignment > Spotify Recommender.ipynb > M+ Spotify Recommender System > # 12. DEMO OF MODEL - PERSONALISED RECOMMENDATIONS BASED ON USER INPUT
+ Code + Markdown | ▶ Run All ⌂ Restart ⌂ Clear All Outputs | Jupyter Variables ⌂ Outline ... Python 3.13.5

# 12. DEMO OF MODEL - PERSONALISED RECOMMENDATIONS BASED ON USER INPUT

def get_user_preferences_interactive():
    """Prompt the user for genre + mood/energy preferences and return a
    profile dict with the same engineered target fields used in training."""
    print("Available genres (sample):", list(spotify["track_genre"].unique()[:15]), "...")
    pref_genre = input("Preferred genre: ").strip()
    while pref_genre not in spotify["track_genre"].unique():
        print("Genre not found, try again.")
        pref_genre = input("Preferred genre: ").strip()

    mood = input("Mood - 'chill', 'energetic', or 'happy'? ").strip().lower()
    mood_map = {
        "chill": {"energy_n_target": 0.25, "valence_n_target": 0.45, "tempo_n_target": 0.3, "danceability_n_target": 0.4},
        "energetic": {"energy_n_target": 0.85, "valence_n_target": 0.6, "tempo_n_target": 0.75, "danceability_n_target": 0.75},
        "happy": {"energy_n_target": 0.65, "valence_n_target": 0.85, "tempo_n_target": 0.6, "danceability_n_target": 0.7},
    }
    targets = mood_map.get(mood, mood_map["chill"])

    profile = {c + "_target": 0.5 for c in norm_features} # neutral default for unspecified features
    profile.update(targets)
    profile["pref_genre"] = pref_genre
    return profile

def build_feature_vector(profile, track):
    """Assemble one row matching feature_cols exactly: raw track features,
    the profile's target features, engineered track-level features, and the
    engineered genre_match / mood_match interaction features."""
    profile_mood = mood_quadrant({
        "valence_n": profile.get("valence_n_target", 0.5),
        "energy_n": profile.get("energy_n_target", 0.5),
    })
    genre_match = 1.0 if track["track_genre"] == profile["pref_genre"] else 0.0
    mood_match = 1.0 if track["mood_quadrant"] == profile_mood else 0.0

    row = ([track[c] for c in norm_features]
           + [profile.get(c + "_target", 0.5) for c in norm_features]
           + [track[c] for c in engineered_track_features])
```

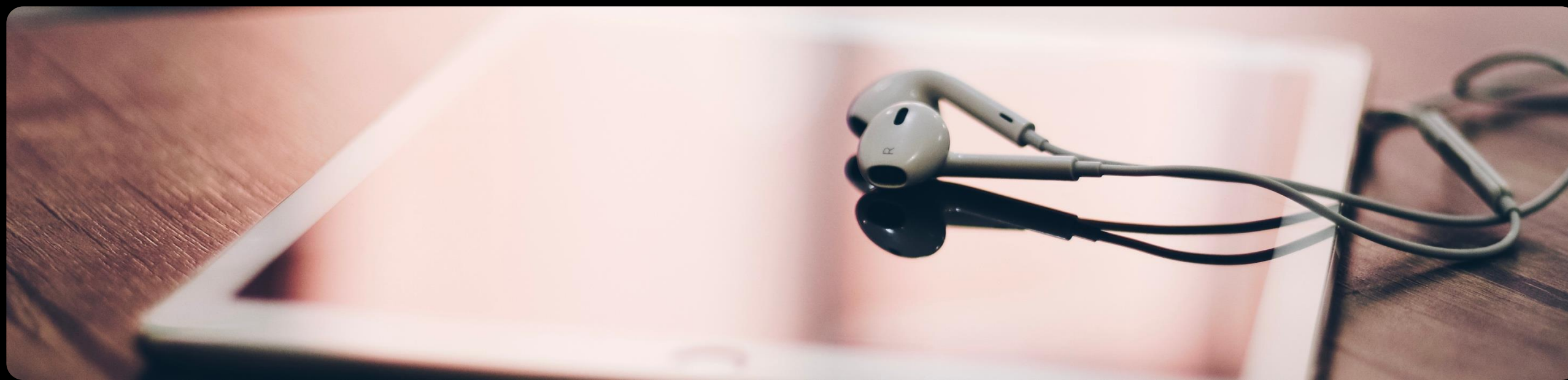




Closing comments



- 🎧 Recommender system using RF model better if prioritizing predictive accuracy and consistency across folds.
- 🎧 MLP model produces fairer recommendations across popularity and genre-rarity.



References

Art. 83 GDPR - General conditions for imposing administrative fines (2018) GDPR.eu. Available at: <https://gdpr.eu/article-83-conditions-for-imposing-administrative-fines/> (Accessed: 20 July 2025).

Borup, D. *et al.* (2023) 'Targeting predictors in random forest regression', *International Journal of Forecasting*, 39(2), pp. 841–868. Available at: <https://doi.org/10.1016/j.ijforecast.2022.02.010>.

Gollapudi, S. *et al.* (2023) 'Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters', *Proceedings of the ACM Web Conference 2023*, pp. 3406–3416. Available at: <https://doi.org/10.1145/3543507.3583552>.

Gorriz, J.M. *et al.* (2024) 'Is K-fold cross validation the best model selection method for Machine Learning?', *Information Fusion*, 135, p. 104404. Available at: <https://doi.org/10.1016/j.inffus.2026.104404>.

Hodson, T.O. (2022) 'Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not', *Geoscientific Model Development*, 15(14), pp. 5481–5487. Available at: <https://doi.org/10.5194/gmd-15-5481-2022>.

Japkowicz, N. and Stephen, S. (2002) 'The class imbalance problem: A systematic study', *Intelligent. Data Analysis*, 6(5), pp. 429–449.

Koehrsen, W. (2020) 'Random Forest Simple Explanation', *Medium*, 18 August. Available at: <https://williamkoehrsen.medium.com/random-forest-simple-explanation-377895a60d2d> (Accessed: 13 July 2026).

Kohavi, R. (1995) 'A study of cross-validation and bootstrap for accuracy estimation and model selection', *Ijcai*. Montreal, Canada, pp. 1137–1145.

Koukaras, P. and Tjortjis, C. (2025) 'Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices', *AI*, 6(10), p. 257. Available at: <https://doi.org/10.3390/ai6100257>.

Kshirsagar, M.M. *et al.* (2025) 'A Multi–Model Machine Learning Framework for Personalised Course Recommendation with Dynamic Feedback', *African Journal of Applied Research*, 11(5), pp. 326–339. Available at: <https://doi.org/10.26437/emg0j607>.



References (cont.)

Lundberg, S.M. and Lee, S.-I. (2017) 'A Unified Approach to Interpreting Model Predictions', *Advances in Neural Information Processing Systems*. Curran Associates, Inc. Available at: <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html> (Accessed: 11 July 2026).

Rathi, M. (2018) *FeedForward Neural Networks*, Mukul Rathi. Available at: <https://mukulrathi.com/demystifying-deep-learning/feed-forward-neural-network/> (Accessed: 11 July 2026).

Sazlı, M.H. (2006) 'A brief review of feed-forward neural networks', *Communications Faculty of Sciences University of Ankara Series A2-A3 Physical Sciences and Engineering*, 50(01), pp. 0–0. Available at: https://doi.org/10.1501/commua1-2_0000000026.

'Spotify — About Spotify' (no date) *Spotify*. Available at: <https://newsroom.spotify.com/company-info/> (Accessed: 12 July 2026).

Spotify Cloud Infrastructure Powers Global Music Streaming (no date) *Sprintzeal Americas Inc.* Available at: <https://www.sprintzeal.com/blog/spotify-cloud> (Accessed: 12 July 2026).
Spotify Tracks Dataset (no date) *Kaggle*. Available at: <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset> (Accessed: 7 July 2026).

